

Relación ejercicios 50

Álvaro García Zafra

Ejercicio 1

alg reloj

4 marcos

en $t=0$ ref pag 2

* $\rightarrow$ bit de referencia con valor 1, si no lo tiene es porque está a 0

X $\rightarrow$ falta de página

P. requerida	1	2	4	6	4	4	3	2	4	1
	2*	2*	2* $\rightarrow$	2* $\rightarrow$	2* $\rightarrow$	2*	3*	3*	3*	3*
	1*	1*	1*	1*	1*	1* $\rightarrow$	1	2*	2*	2*
$\rightarrow$	$\rightarrow$		4*	4*	4*	4*	4 $\rightarrow$	4 $\rightarrow$	4*	4
		$\rightarrow$		6*	6*	6*	6	6	6 $\rightarrow$	1*
	X		X	X			X	X		X

P. requerida	1	3	2	3	5	5	5	3	5	1	4	2
	3*	3*	3*	3*	3	3	3	3*	3*	3*	3	3
	2*	2*	2*	2*	2	2	2	2	2	2	4*	4*
	4	4	4	4	5*	5*	5*	5*	5*	5*	5*	5
	1	1*	1*	1*	1*	1	1	1	1	1	1	2*

X

X X

En total se producen 9 faltas de página

El funcionamiento del algoritmo de reloj se basa en un FIFO con segunda oportunidad, para esto antes de echar al más antiguo mira si tiene su bit de referencia a 1 o 0, en el caso de tenerlo a 0 lo puede echar, mientras que si lo tiene a 1 cambia este bit a 0 y mueve el puntero al próximo.

Si la página requerida ya se encuentra en memoria pone su bit de referencia a 1 si está en 0 y en el caso de que una página ya esté referenciada el puntero se queda en el lugar que estaba, ~~pero~~ incluso cuando ~~aumenta~~ se modifique el bit de referencia a 1, el puntero solo se mueve cuando haya faltas de página.

Ejercicio 2

No, no siempre va a ser preferible usar DMA, aunque sea más avanzado. Si por ejemplo la cantidad de datos a transferir es muy pequeña, un carácter, ~~por~~ la CPU va a tardar más tiempo en la configuración del DMA que el tiempo que tardaría en dicha interrupción.

El DMA será superior y recomendado cuando haya que transferir grandes volúmenes de datos a alta velocidad, mientras que para tareas pequeñas o dispositivos lentos la sobrecarga de configurarlo no compensa.

Ejercicio 3

8 índices directos, 2 ind simples (1 nivel), 1 ind doble (2 niveles), 2 ind triples (3 nivel)

tam bloque = 3kB

puntero = 64b = 2^6 b = 2^3 B

$$\text{entradas / índice} = \frac{3 \cdot 2^{10}}{2^3} = 3 \cdot 2^7 = 384 \text{ ~~dir / bloque~~ }^{\text{entradas}}$$

1. ¿d bloques fichero 2000kB?

$$BL = \frac{2000 \cdot 2^{10}}{3 \cdot 2^6} = 666'6 \approx 667 \text{ bloques}$$

A continuación vamos a calcular cuantos índices necesitaremos:

$$\frac{BL}{8} = 667 - 8 = 659 - 384 = 275 - 384$$

Un fichero de 2000kB necesitará 2 bloques índice, serán 2 índices simples (1er nivel)

2. máx que puede alcanzar un arch

$$8 \text{ entradas} + 2 \cdot 384 + 384^2 + 2 \cdot 384^3 = 113394440 \text{ entradas}$$

$$\text{Como cada una tiene un tam de 3kB: } 113394440 \cdot 3 \text{ kB} = 340183320 \text{ kB} = 324'42 \text{ GB}$$

El tam máx de archivo que soporta es 324'42GB

Ejercicio 4

Gestión por interrupciones

P1 ejecuta CPU $t=0 \rightarrow t=10$, una vez termina va a ejecutar la op E/S (15ms) 10-25

P2 ejecuta CPU $t=10 \rightarrow t=20$, una vez termina se ejecuta la op E/S (15ms) 20-35

Entre el instante 20 y 25 ambos procesos están bloqueados.

En el ~~instante~~ instante 25 ~~P1~~ termina la E/S de P1 y ~~en~~ la CPU consume los 0'4ms de la interrupción.

Una vez terminada la interrupción P1 vuelve a ejecutar, entre los instantes 25'4 y 35'4 y termina tras esto

P2 termina en el instante 35 pero la CPU está ocupada con la interrupción de P1, por lo que en el instante 35'4 recupera el control, genera la interrupción, ~~que~~ que termina en el instante 35'8 y se ejecuta su segunda ráfaga, 35'8 - 0 45'8

El desperdicio sería los 5ms que ha estado esperando a que ~~la proce~~ el proc 1 dejase de estar bloqueado + los 0'4ms de cada una de las 2 interrupciones producidas

Espera ocupada

P1 ejecuta su primera ráfaga $0 \rightarrow 10$

Cuando termina se queda en la CPU ejecutando un bucle de espera, sin hacer nada, 15ms

P1 ejecuta su segunda ráfaga, ya que no llegó a salir de la CPU, $25 \rightarrow 35$ y termina.

Cuando termina P1, P2 realiza el mismo proceso.

P2 ejecuta su primera ráfaga $35 \rightarrow 45$ y al terminar se queda ejecutando el bucle de espera, $45 \rightarrow 60$

Al terminar la espera ejecuta su segunda ráfaga $60 \rightarrow 70$ y P2 termina

Como podemos ver el desperdicio en este caso es de $15\text{ms} + 15\text{ms}$, por lo que se puede apreciar que se está desperdiciando muchísima más CPU en el modo de Espera Ocupada.

Ejercicio 5

Este fallo ~~se~~ consistió en un backdoor en el que se introdujo código malicioso en una librería linux. Dicho código se ocultó en archivos de pruebas y modificaba el control SSH, permitiendo la conexión remota saltando la autenticación.

Este ~~problema~~ problema de seguridad es lógico, ya que manipula el código fuente y es un backdoor, ya que su misión principal era permitir el acceso no autorizado al sistema.

Para evitar esto se puede implementar:

Política de seguridad: revisión del código fuente antes de que este sea actualizado, permitiendo detectar vulnerabilidades antes de que lleguen a los dispositivos.

Mecanismo de seguridad: para permitir la política establecida puede establecerse que se necesite la firma de otros 2 desarrolladores que cuando ambos revisen el código y vean que no se encuentran vulnerabilidades permitan que se actualice el código y se vaya a la bre.